FinIntell AI — Technical Architecture (HLD & LLD)

System Topology, Data Flow, ERD Schema & Component Hierarchy

> **Architectural Scope:** This document provides an exhaustive High-Level Design (HLD) and Low-Level Design (LLD) specification for the **FinIntell AI SaaS Platform**. It establishes system topology, multi-tier boundaries, data flow pipelines, PostgreSQL ERD schemas with pgvector extensions, sequence workflows, and Spring Boot component hierarchies.

1. High-Level Design (HLD): System Topology & Tier Architecture

The FinIntell AI platform utilizes a 3-tier micro-service ready architecture built around a decoupled **React 19 Single Page Application (SPA)** frontend and a stateless **Java 21 / Spring Boot 3** application backend. All requests are filtered through a centralized API Gateway and Security Guard layer.

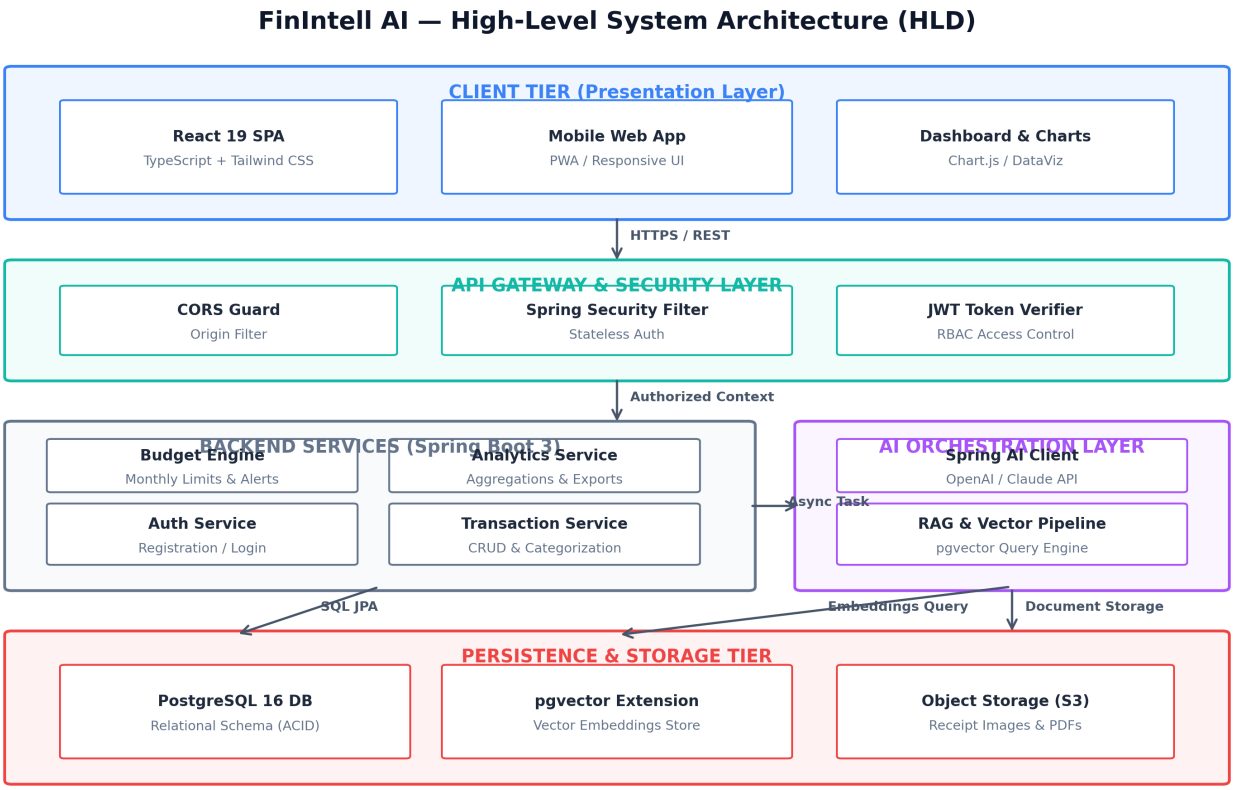


Figure 1.1: FinIntell AI Multi-Tier High-Level System Architecture Diagram

Architecture Tier	Technology Stack	Primary Functionality & Guardrails
Presentation Tier	React 19, TypeScript, Tailwind CSS, Chart.js	Responsive dashboard UI, state management, client-side validation, and interactive financial data visualizations.

Gateway & Security Tier	Spring Security 6, JWT, CORS Filter	Stateless authentication filter, RBAC claims verification, rate limiting, and CORS origin enforcement.
Core Application Services	Java 21 LTS, Spring Boot 3.3, Spring Data JPA	Business logic execution, multi-category expense rules, budget alert calculations, and REST fulfillment.
AI Orchestration Layer	Spring AI Framework, OpenAI / Claude API, pgvector	Receipt document OCR ingestion, structured JSON parsing, prompt engineering, and semantic RAG search.
Persistence & Object Storage	PostgreSQL 16, pgvector Extension, AWS S3	ACID transactional storage, vector embedding similarity search, and cloud file staging.

2. High-Level Data Flow Design (DFD Level 1: Receipt AI Parsing)

Data flows asynchronously through the system when processing uploaded receipts. The file is staged in cloud storage while Spring AI delegates image OCR and entity extraction to an LLM before validating the JSON schema and persisting structured entities to PostgreSQL.

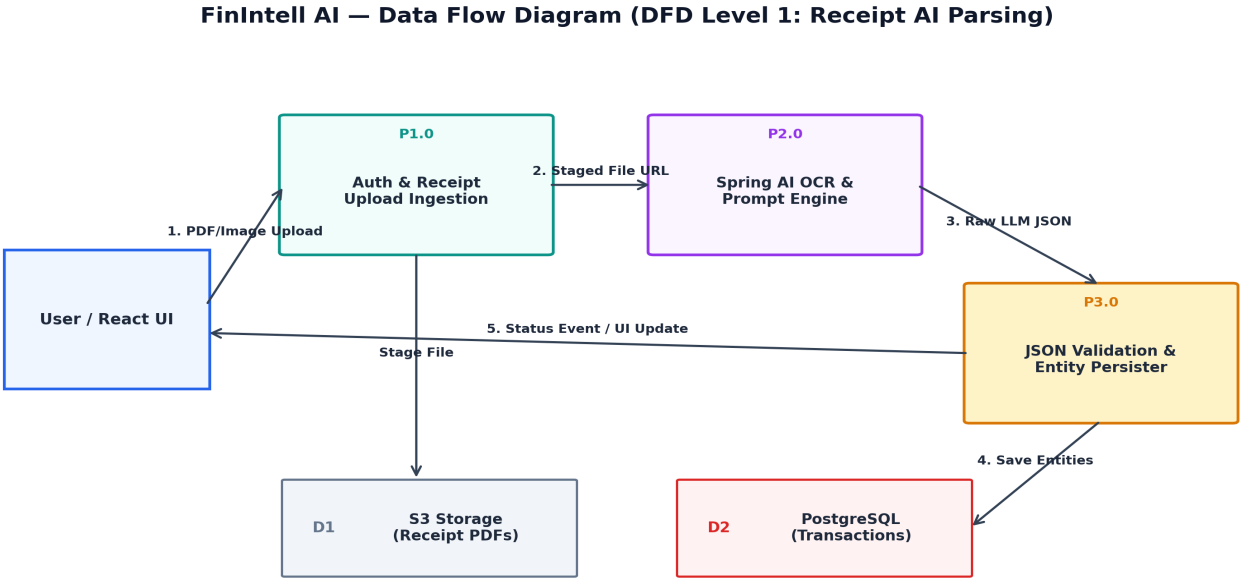


Figure 2.1: Data Flow Diagram (DFD Level 1) for Asynchronous AI Receipt Ingestion

3. Low-Level Design (LLD): Database Schema & Entity-Relationship Diagram

The relational database is constructed in **PostgreSQL 16** with normalized Third Normal Form (3NF) relational tables alongside a high-dimensional vector embedding column powered by the pgvector extension for natural language RAG querying.

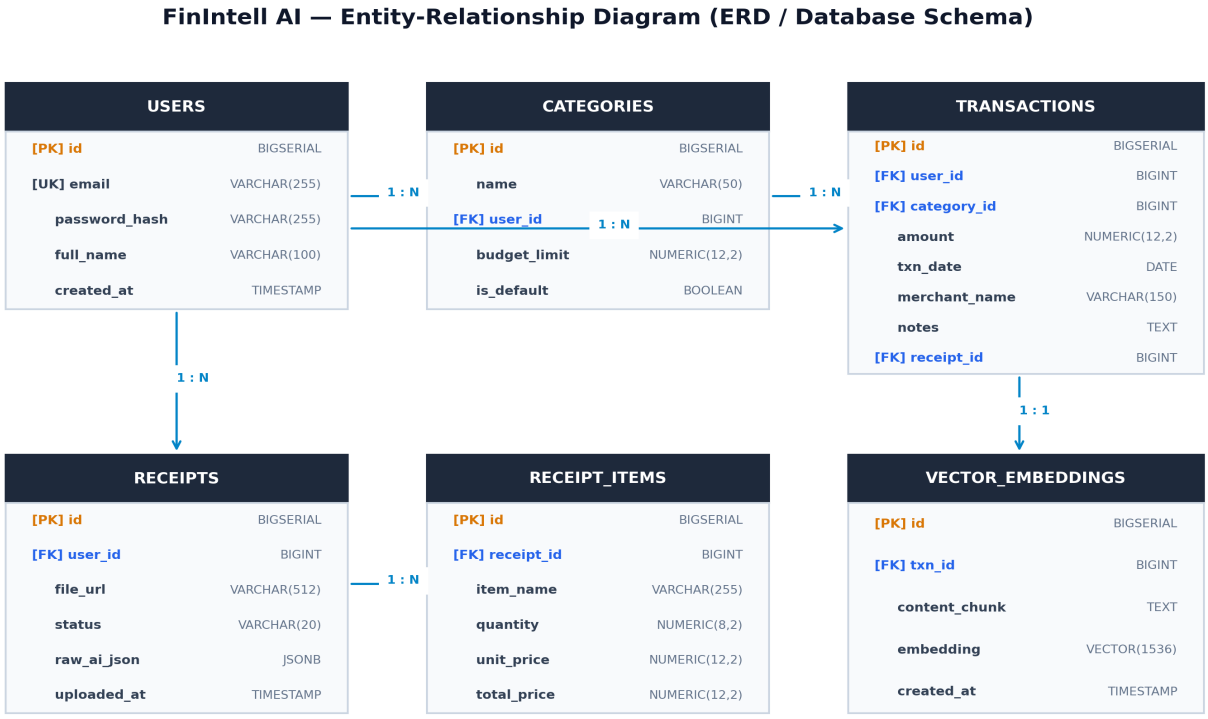


Figure 3.1: Complete Entity-Relationship Diagram (ERD) with Primary/Foreign Keys and Types

Table Name	Primary Key	Foreign Keys	Special Indexing & Constraints
USERS	id (BIGSERIAL)	None	UNIQUE(email), B-Tree Index on email.
CATEGORIES	id (BIGSERIAL)	user_id -> USERS(id)	B-Tree Index on (user_id, name).
TRANSACTIONS	id (BIGSERIAL)	user_id, category_id, receipt_id	Composite Index on (user_id, txn_date DESC).
RECEIPTS	id (BIGSERIAL)	user_id -> USERS(id)	CHECK (status IN ('PENDING', 'COMPLETED', 'FAILED')).
RECEIPT_ITEMS	id (BIGSERIAL)	receipt_id -> RECEIPTS(id)	ON DELETE CASCADE on receipt_id.
VECTOR_EMBEDDINGS	id (BIGSERIAL)	txn_id -> TRANSACTIONS(id)	HNSW Index using Cosine Distance (vector_cosine_ops).

4. Low-Level Design (LLD): Async Sequence Diagram & Behavioral Logic

To guarantee high throughput and sub-50ms API responsiveness, receipt uploads return an immediate receipt token with a PENDING state while background thread pools execute OCR prompting, entity mapping, and vector embedding generation.

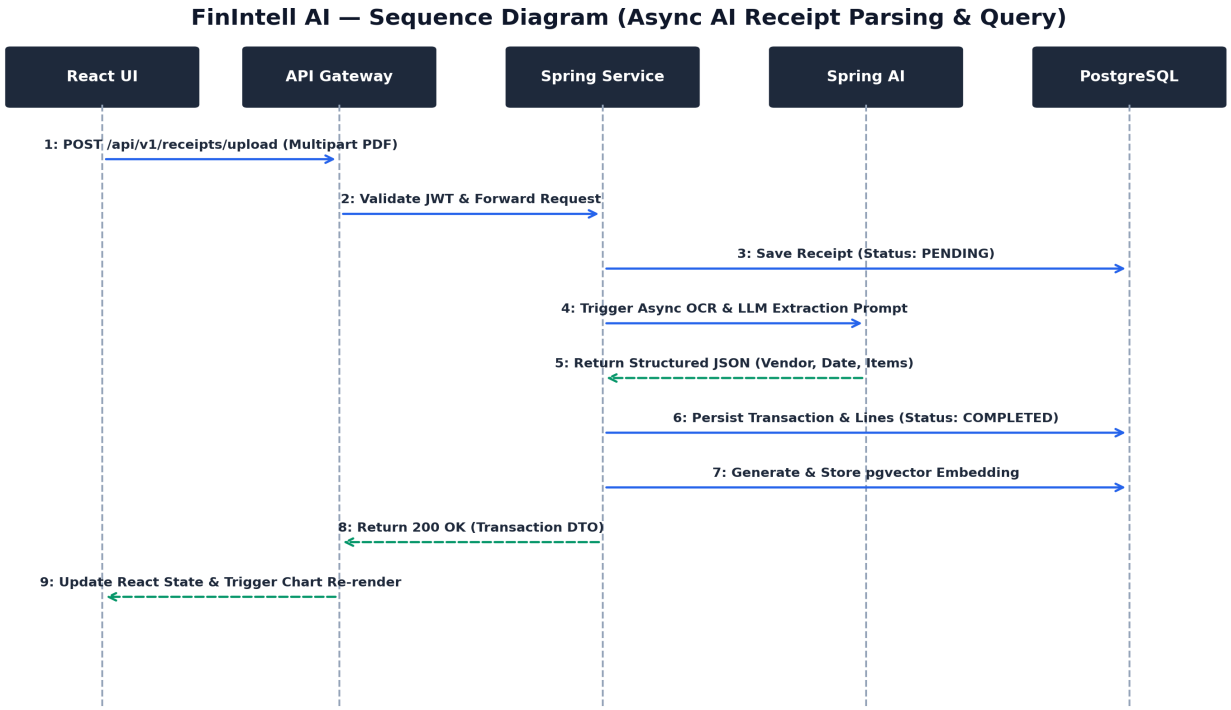


Figure 4.1: Sequence Diagram detailing Step-by-Step System Communication and State Management

5. Low-Level Design (LLD): Spring Boot Class & Module Hierarchy

The Spring Boot backend enforces strict separation of concerns using a layered pattern: REST Controllers handle request serialization, Service interfaces declare transactional boundaries, and Spring Data Repositories manage database interactions.

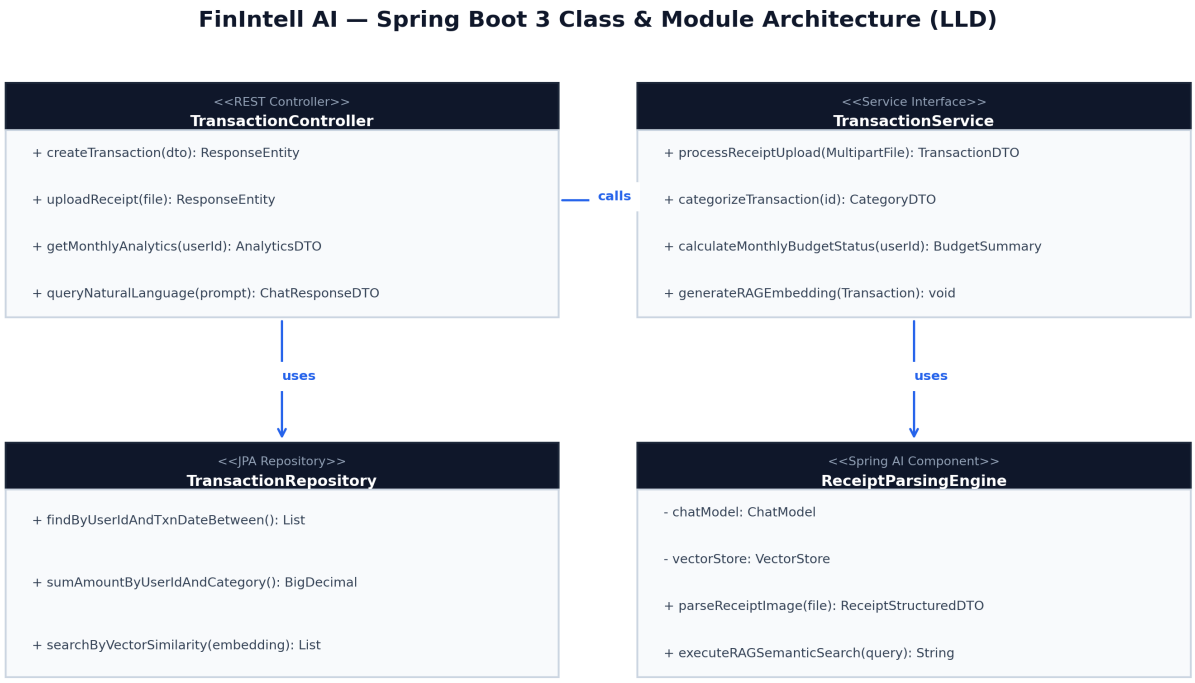


Figure 5.1: Class Diagram displaying Spring Boot 3 Layered Architecture and Spring AI Injection

6. Low-Level Design (LLD): REST API Endpoint Specifications

HTTP Method	Endpoint Path	Request Body / Params	Success Response
POST	/api/v1/auth/login	AuthRequestDTO (email, password)	200 OK -> AuthResponseDTO (jwtToken, user)
GET	/api/v1/transactions	page, size, categoryId, startDate	200 OK -> Page
POST	/api/v1/transactions	CreateTransactionDTO	201 Created -> TransactionResponseDTO
POST	/api/v1/receipts/upload	MultipartFile file	202 Accepted -> ReceiptUploadResponseDTO
POST	/api/v1/ai/chat-query	ChatQueryDTO (promptText)	200 OK -> ChatAnswerDTO (response, sources)